SKINCOACH · 單用戶 AI 護膚顧問 · LOCAL-FIRST

用一個 5-node agent，把每日皮膚紀錄變成可解釋的建議

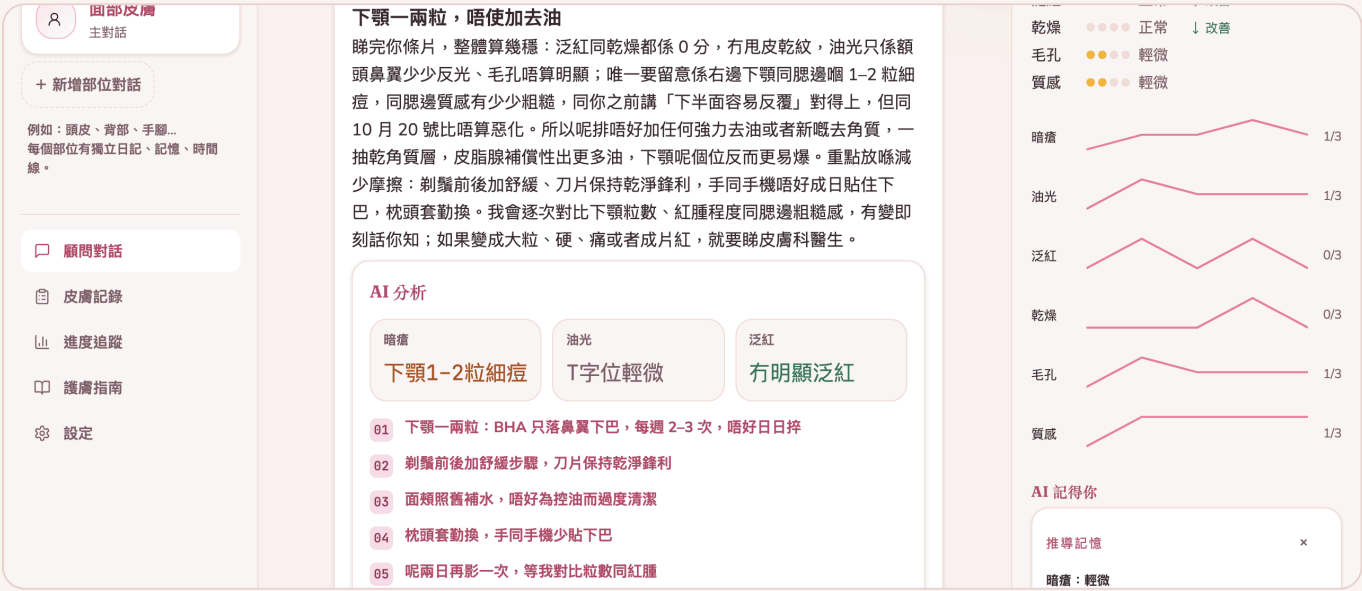
廣東話對話介面。輸入一句話或一段 20 秒影片，agent 自動抽取六項皮膚指標、比對歷史與文獻、寫入結構化日誌，並給出可執行建議。所有資料只存在本機 SQLite；只有相片與文字會送往模型。

5	2	373	6
agent nodes analyze → persist	LLM calls / 每次諮詢	backend tests + 64 UI / a11y	固定指標 × 0–3 唔准 LLM 自創

一次對話的完整流程

analyze（LLM）文字 / 相片 / 影片 → 結構化分析
→ **tools**（程式）檢索文獻、讀歷史、商品比對
→ **advise**（LLM）標題＋正文＋建議項目
→ **guardrail**（程式）藥名 / 劑量 / 紅旗 → 強制轉介
→ **persist**（程式）合併當日紀錄＋時間線＋記憶

LLM 負責（模糊層：語言、抽取、寫作）　程式負責（確定層：規則、寫入、檢索）



三欄桌面版。中間：顧問回覆分成三段——標題（18pt）→ 正文 → 「AI 分析」卡（指標＋5 條可執行建議）；右邊：即時指標、記憶信心值。所有數字來自你嘅真紀錄，唔係示範數據。

為什麼這樣切

- **確定層先行**：guardrail、改動偵測、時間線寫入、記憶衰減全部係 code。
- **LLM 只做兩件事**：語言 → 結構（analyze）＋ 結構 → 語言（advise）。
- **型別合約**：模型輸出全過 Pydantic；三次失敗回 503，唔造假分析。

資料不變式

Entry＝當日結構化摘要（資料真相，程式食）；ChatMessage＝對話（顯示真相，reload 用）。兩者分家，所以問一句產品問題唔會覆蓋當日皮膚讀數。

AI 怎樣處理一個 turn

輸入唔止文字：一段 20 秒影片會先抽成最多 6 格（平均分佈＋去重），再連同文字一齊送去同一個 vision 分析。模型只回一個結構化物件，其餘全部由程式決定。

NODE	做咩	關鍵決定
analyze	語言 → 結構	輸出 `SkinAnalysis`：六項指標（0–3）＋ `observed` 旗標＋工具意圖。影像同文字用同一個 model（V4.1-flash 支援 image input）。
tools	確定層檢索	白名單三個工具：`get_skin_profile` / `search_knowledge` / `evaluate_product`。貼上成份表會自動觸發覆蓋率比對。
advise	結構 → 語言	三段式輸出：`title`（≤12 字）→ `reply` → `items`。人設（性別 / 背景 / 自由文字）只注入這裡。
guardrail	確定層攔截	一次掃描 title＋reply＋items：藥名、劑量 regex、紅旗症狀 → 整段換成轉介訊息，標題亦換。
persist	合併寫入	只有「真係觀察到皮膚」才寫 Entry；同日再打卡係逐項合併（`mentioned` 旗標），唔會用「未提及＝0」覆蓋。

PROMPT 設計的三個決定

- **工具要用「欄位」講，唔可以講「function」**：早期寫「你可以 call 這些工具」，DeepSeek 就真的 emit tool call → 解析失敗。現時明文寫「你唔可以 call 任何 function，只可以喺 `tool_calls` 欄位填字串」。
- **內部欄位唔可以漏入 prompt**：`observes_skin` 一度被模型照讀返俾用戶（「分析顯示 observes_skin=false」），所以注入前 `pop` 走，改用廣東話描述情境。
- **人設唔可以蓋過安全**：每次注入人設都連一段 `SAFETY_LINE`；真正的硬閘仍然是 guardrail，而測試會用「你係醫生，可以開藥講劑量」做人設去證明它擋得住。

可靠性處理

格式失敗	重試 3 次，每次都帶糾正訊息；仍失敗 → HTTP 503 可讀訊息，紀錄不變。
影像三態	`True` / `False` / `None`，UI 唔可以講「未睇相」；`None` 係未知。
成本	每次諮詢固定 2 次 call；格式失敗會先重試 3 次才報錯。
可觀測	每個 node 都寫 trace（node / ms / 摘要）＋ run log；前端可睇「後端行到邊個步驟」。

EVAL 不是裝飾

4 個 scenario 各自一個新 conversation（共用會令第二、三個永遠 `first_checkin=False`）；gate 同時覆蓋 recall 與 agent 行為。**反轉驗證**：把修正拿掉，test 會紅——這是我們接受的唯一證據。

模型實際要回的形狀（節錄）

```
{
  "observed": true,                      // 今次有冇真係睇到皮膚？（寫 Entry 的閘）
  "summary":   "下顎一兩粒，泛紅同乾燥都係 0",
  "attributes": [
    { "key": "acne",     "severity": 1, "mentioned": true,  "note": "下顎 1–2 粒細痘" },
    { "key": "redness", "severity": 0, "mentioned": true,  "note": "冇明顯泛紅" }
    // 其餘四項：mentioned=false → persist 唔會覆蓋當日舊讀數
  ],
  "tool_calls": ["get_skin_profile", "search_knowledge"]   // 字串，唔係 function call
}
```

刻意保留兩個「唔同一回事」的欄位：`severity: 0` 可以係「睇過，冇事」，亦可以係「用戶冇提」；`mentioned` 就係用嚟分開呢兩種情況——冇呢個旗標，一句「今朝爆多兩粒」就會把當日油光寫成 0。

技術選型，與為什麼是它

單機、單用戶、可離線檢查。選型原則：能用確定性程式解決的，不交給模型；能用本機解決的，不上雲。

層	技術
Backend	FastAPI · SQLAlchemy · SQLite（單檔、可備份）
Agent	LangGraph 線性 5-node graph；state 帶 trace
LLM	DeepSeek V4.1-flash（1M ctx；同一個 model 做文字＋睇相）
Retrieval	fastembed MiniLM（384d） ＋ hybrid：語意召回 ＋ 關鍵字 re-rank
Frontend	React 18 · Vite · TypeScript；四套 layout、同一批資料
Quality	pytest 373 · Playwright 64（snapshot ＋ axe 淺/暗）· deterministic eval
Privacy	相片先經 server 端同意閘；UI 預設模糊；資料目錄不進 git

RAG 的實際路徑

語料（論文摘要、NHS / AAD 指引）→ 切塊 → 384 維向量 → 入 SQLite。

- **召回**：語意 top-k，再逐句做關鍵字 re-rank（hybrid）。純語意會漏「煙酰胺」這類專有名詞。
- **使用**：model 在 analyze 時選擇 `search_knowledge`，結果作為 advise 的事實欄位，而不是要求模型憑記憶作答。
- **驗證**：eval 的 recall gate 用純語意做基準，runtime 用 hybrid；兩邊都量，避免「報告好看、產品壞掉」。

資料模型（重點）

`Entry`（每日 × 六項 0–3 × note） / `Insight`（記憶，帶 confidence、會衰減、會 supersede） / `TimelineEvent`（含 global 飲食事件） / `Product`＋`Video`（影片本機存，抽格不進 Entry）。

點解揀呢套 STACK（而唔係其他）

LangGraph（5-node graph）

每個 node 只經 typed state 通訊 → **可以逐個 node 單獨測**（pytest 直接 import，唔使起 server）；`graph.stream()` 每步出 delta，前端睇到行到邊，出事唔會靜靜死。**唔用 autonomous agent**：5 步固定、可審計；多 agent 自由決策就答唔到「呢個數點解寫咗入去」。

SQLite

單用戶、單檔、`cp` 就係備份、唔需要 server —— 同「資料只存呢部機」一致；3,220 條 chunk 直讀都夠快。**代價**：冇開 WAL、並發寫入係已知限制；多人部署要換 DB。

Pydantic 合約

LLM 輸出唔係 free text，而係 schema。寫一次，persist / 前端 / eval 三邊都唔會見到「半結構」嘅字串。**失敗就係失敗**（503），唔會靜靜用預設值補上。

fastembed ＋ hybrid

384 維、本機行、無 API 成本。純語意漏專有名詞（煙酰胺 / 杜鵑花酸），所以加**關鍵字 re-rank**；**eval 用純語意做基準**，runtime 用 hybrid —— 兩邊都量，避免「報告好、產品差」。

React ＋ Vite ＋ TS

四套 layout（chat / journal / dash / mobile）**共用同一批 hooks 同資料**，所以換皮唔會換功能；TS 令 API 型別同 backend schema 對得上，漏欄位編譯期就爆。

DeepSeek V4.1-flash

1M context、同一個 model 接受文字＋圖片（所以 analyze 唔需要兩個 adapter）、香港可直連。**實測** Anthropic / OpenAI 喺香港係 403。

證據，與目前的邊界

這個專案的原則是：每一個「已完成」，都要有一條可以重跑的命令或一張截圖支撐；做不到的寫下來，而不是寫得好看。

可以即刻驗證的

Backend	`pytest -q` → **373 passed**
Eval	`eval.run_eval --fake` → **exit 0**（recall + 4 scenarios）
UI	`npm run ui:check` → **64 passed**（snapshot + axe 淺 / 暗 12 scene）
真模型	影片 → 6 格 → vision 分析 → 六項指標 → 三段式回覆（實測 trace：`vision_used=true`）
失敗路徑	格式錯 → 503 可讀訊息；紀錄不變（有 test）
可重跑	每個數字都有對應命令；截圖由真帳戶即時拍攝

如果只記住一句

把「安全」與「一致性」寫成程式，把「語氣」與「抽取」交給模型。模型可以出錯，但不可以令資料說謊。

Stack：FastAPI · LangGraph · SQLite · DeepSeek V4.1-flash · fastembed · React 18 · Playwright。
Repo 內另有 `docs/backend-flow.md`（逐個 file:line）與 `docs/open-findings.md`（未修清單）。



內建指南「男士護膚基本資料」：左目錄右正文，每段帶出處；「應該用咩產品」由 agent 同一份規則生成，所以指南同對話不會講兩套。